

Operations Architecture

Document: docs/architecture/OPERATIONS_ARCHITECTURE.md

System: HackNest — Student Operating System

Status: Authoritative — reconciled against README.md, MASTER_ARCHITECTURE.md, DATABASE_ARCHITECTURE.md, SECURITY_ARCHITECTURE.md, DEPLOYMENT_ARCHITECTURE.md

Type: Architecture Specification Only — No Implementation, No Scripts, No GitHub Actions, No Monitoring Configuration, No Infrastructure Code

This document follows README.md's index and MASTER_ARCHITECTURE.md's governance exactly, occupying the position README.md already reserved for it in the Subsystem architecture layer. Per MASTER_ARCHITECTURE.md Section 9 (Cross-Document Precedence), this document is the **sole authoritative owner** of day-to-day operational execution, on-call/escalation structure, and incident handling in practice. It explicitly does **not** own: numeric RPO/RTO targets or infrastructure/backup mechanics (owned by DEPLOYMENT_ARCHITECTURE.md), the secrets management policy itself (owned by SECURITY_ARCHITECTURE.md), or the audit log record specification (owned by DATABASE_ARCHITECTURE.md). Where this document touches those concerns, it defines the *operational execution* of them and cites the owning document for the *policy or specification*.

Note on DEPLOYMENT_ARCHITECTURE.md's reconciliation status: at the time of authoring this document, DEPLOYMENT_ARCHITECTURE.md still reflects its original standalone draft (authored before MASTER_ARCHITECTURE.md and DATABASE_ARCHITECTURE.md existed) and has not yet been through the same reconciliation pass applied to SECURITY_ARCHITECTURE.md. This document cross-references it as-is and flags, in the Validation Report below, that a future reconciliation pass on DEPLOYMENT_ARCHITECTURE.md may require a corresponding follow-up revision here.

Purpose

A Student Operating System does not stop being architecture once it is deployed — it has to keep running, correctly and predictably, every day a student might need it. Operations is the discipline of making sure the system described by every other document in docs/architecture/ actually stays true in production over time: that a release goes out safely, that a failure is noticed before a student notices it first, that a bad night doesn't become a bad week, and that the people responsible for the system know exactly what to do when something goes wrong.

The operational philosophy here is the same as the rest of this documentation set: specify before building (`MASTER_ARCHITECTURE.md` Section 2), and treat every operational responsibility as owned by exactly one document, so that "who handles this" is never ambiguous when it matters most — during an incident. This document is that specification. It does not implement a single script, dashboard, or pipeline.

Operational Principles

- **Reliability.** The system behaves predictably under both normal and adverse conditions; operational practice exists to keep it that way, not just to react when it isn't.
 - **Availability.** Students can reach HackNest when they need it; availability is treated as a continuously tracked property (Section: Monitoring), not an assumption.
 - **Scalability.** Operational practice anticipates growth (Section: Capacity Planning) rather than discovering scaling limits during an incident.
 - **Observability.** Every layer of the running system produces enough signal to know its health without guessing (Section: Monitoring, Logging).
 - **Automation.** Routine operational tasks are automated wherever reliable automation is possible; manual effort is reserved for judgment calls, not repetitive execution (consistent with `MASTER_ARCHITECTURE.md` Section 2's principles applied operationally).
 - **Maintainability.** Operational knowledge is written down (Section: Documentation) rather than held only in individual memory, so the system remains operable even as the team changes.
 - **Disaster Recovery.** Catastrophic failure is planned for in advance, not improvised during one (Section: Backup & Recovery).
-

Production Operations

- **Daily operations.** Routine operational activity — health check review, alert triage, backlog of non-urgent operational findings — follows a defined daily cadence owned by whoever holds on-call responsibility (see Alerting section) for that period, so nothing is implicitly "someone else's job."
- **Release operations.** Every release follows the environment promotion path and Release Quality Gates already defined in `TESTING_STRATEGY.md` and `DEPLOYMENT_ARCHITECTURE.md`; this document's operational contribution is the **execution checklist** the on-call/release-responsible person actually runs through at release time (see Documentation section for where that checklist lives) — the gates themselves are not redefined here.

- **Incident management.** Follows the Incident Response Process owned by `SECURITY_ARCHITECTURE.md` Section 23 for security-classified incidents, and this document's own Alerting/Escalation structure (below) for the operational mechanics of any incident (security-classified or not) — staffing, who is notified, and in what order.
 - **Maintenance windows.** Planned, communicated in advance, and scheduled to minimize disruption to predictable high-usage periods (e.g., avoiding a hackathon's active submission window); executed with the same Rollback Capability expectation as any other change, per `DEPLOYMENT_ARCHITECTURE.md`'s Deployment Principles.
 - **Rollback procedures.** The technical rollback mechanism itself is owned by `DEPLOYMENT_ARCHITECTURE.md`'s Release Strategy; this document owns the **operational decision process** — who is authorized to trigger a rollback, and under what observed conditions (see Alerting's Critical tier) a rollback is the default response rather than a forward fix.
-

Monitoring

Monitoring infrastructure and tooling mechanics are owned by `DEPLOYMENT_ARCHITECTURE.md`'s Monitoring & Observability section. This document defines the **categories that must be monitored** and who is operationally responsible for each:

- **Application monitoring.** Request rates, error rates, and latency per domain (per `API_ARCHITECTURE.md`'s domain list); reviewed as part of daily operations and the first signal checked during an incident.
- **API monitoring.** Conformance to the response-time targets that `TESTING_STRATEGY.md` designates `TESTING_STRATEGY.md` itself as owning; this document's role is ensuring those targets are actually watched continuously in production, not only validated pre-release.
- **Database monitoring.** Query performance, connection pool health, and replication/backup health (the latter cross-referencing `DEPLOYMENT_ARCHITECTURE.md`'s Infrastructure section) — distinguishable from application-level monitoring so a database-layer issue is never mistaken for an application bug.
- **Authentication monitoring.** Sign-in success/failure rates and session-issuance health, feeding directly into the Security Logs and Failed Login Monitoring already defined in `SECURITY_ARCHITECTURE.md` Section 20 — this document's addition is that authentication health is tracked as its own distinct operational category, not folded into general application monitoring.

- **Storage monitoring.** Upload success/failure rates and storage capacity trends, feeding Capacity Planning below.
- **Performance monitoring.** System-wide latency and throughput trends over time, distinguishing a gradual regression from a sudden incident.
- **Business metrics.** Non-security, non-infrastructure signals that indicate product health (e.g., registration completion rate, active team formation rate) — owned operationally by this document for *visibility*, while the underlying data itself remains owned by the Analytics domain per `DATABASE_ARCHITECTURE.md` Section 22.

Logging

Log **infrastructure** (where logs are shipped, how they're queried) is owned by `DEPLOYMENT_ARCHITECTURE.md`. This document owns the **operational categories and retention discipline**:

- **Application logs.** General request/response and application-behavior logs, used for day-to-day debugging; lowest sensitivity, shortest retention of the categories here.
- **Security logs.** The distinct stream defined in `SECURITY_ARCHITECTURE.md` Section 20 (failed authentication, authorization failures, permission changes); operationally reviewed on a defined cadence as part of Access Reviews (see Operational Security below), not only during an active incident.
- **Audit logs.** The permanent record specified in `DATABASE_ARCHITECTURE.md` Section 9; this document's operational responsibility is ensuring audit logs are actually reviewed periodically (see Operational Security), not merely stored and forgotten.
- **Error logs.** Captured with full internal diagnostic detail server-side (per `API_ARCHITECTURE.md`'s Error Handling), reviewed as the first artifact during incident investigation.
- **Access logs.** Who accessed what, at the infrastructure/request level, distinct from Audit Logs' higher-level "what action was taken" record — access logs answer "was this endpoint reached," audit logs answer "what did that reaching accomplish."
- **Log retention policy.** Mirrors the retention-category model established in `DATABASE_ARCHITECTURE.md` Section 15: Audit Logs retained permanently (no operational override permitted); Security Logs retained long enough to support a delayed-discovery incident investigation; Application/Error/Access logs retained for a bounded, shorter operational window sufficient for debugging and trend analysis. Exact numeric windows for the shorter-retention categories are a placeholder pending `MASTER_GOVERNANCE_AMENDMENT_001.md`'s operational guidance, flagged rather than invented, consistent with this workstream's established practice.

Alerting

- **Critical alerts.** Conditions indicating active, significant student-facing impact or a security event in progress (e.g., authentication system down, database unreachable, a Critical-severity finding per `TESTING_STRATEGY.md`'s Bug Severity scale actively occurring in production). Critical alerts page the on-call responder immediately and are the default trigger for the rollback decision process above.
- **Warning alerts.** Conditions indicating degraded but not fully broken service, or a trend heading toward a Critical threshold (e.g., elevated but not yet failing error rates, approaching storage capacity). Reviewed promptly but do not require immediate off-hours response by default.
- **Informational alerts.** Non-urgent signals useful for trend awareness (e.g., a routine backup completed, a deployment succeeded). Not paged; reviewed during daily operations.
- **Escalation process.** A Critical alert not acknowledged within a defined window automatically escalates to a secondary responder, then to the interim approval authority defined in `MASTER_ARCHITECTURE.md` Section 5, ensuring no Critical condition silently goes unaddressed due to a single unavailable individual.
- **On-call responsibilities.** The on-call responder is the first point of triage for any Critical or Warning alert during their rotation, holds the authority to trigger a rollback per the Production Operations section above, and is responsible for either resolving the issue or explicitly handing it off — an alert is never left in an ambiguous "someone's probably looking at it" state.

Backup & Recovery

- **Ownership split (explicit, to avoid re-introducing the duplication the Architecture Audit identified):** the numeric Recovery Point Objective (RPO) and Recovery Time Objective (RTO) targets, and the underlying backup/restore infrastructure mechanics, are owned entirely by `DEPLOYMENT_ARCHITECTURE.md`. This document does not restate or redefine those numbers — it owns only the **operational execution and verification** of the process built around them.
- **Backup schedule.** Executed per the frequency `DEPLOYMENT_ARCHITECTURE.md` defines; this document's responsibility is confirming, operationally, that scheduled backups actually completed (an Informational alert per the Alerting section above), not re-deciding how often they should run.
- **Recovery process.** The technical restore steps are owned by `DEPLOYMENT_ARCHITECTURE.md`; this document owns **who is authorized to initiate a restore** (an `admin / service`-tier action per `SECURITY_ARCHITECTURE.md` Section 21) and the operational communication process during a live recovery.

- **RTO / RPO.** Cited from `DEPLOYMENT_ARCHITECTURE.md`, not redefined here. As of this document's authoring, those numeric targets remain a flagged placeholder in that document pending `MASTER_GOVERNANCE_AMENDMENT_001.md`'s operational guidance — this document inherits that same open flag rather than inventing its own numbers.
 - **Disaster recovery testing.** Owned operationally by this document: restore procedures are rehearsed on a defined cadence against a non-production environment, with results documented (see Documentation section) so that restore capability is verified ahead of an actual incident rather than assumed. Exact testing cadence is likewise a placeholder pending governance guidance.
-

Capacity Planning

- **User growth planning.** Tracked against the one-million-student target established in `MASTER_ARCHITECTURE.md` Section 15, using the Business Metrics monitoring category above as the leading indicator for when infrastructure scaling conversations need to start, well before capacity is actually exhausted.
 - **Database growth.** Tracked against the partitioning-pressure domains already identified in `DATABASE_ARCHITECTURE.md` Section 13 (Messages, Audit Logs, Notifications, Activity Feed) — this document's role is operational trend-watching of those specific domains' growth, feeding a decision point for when `DATABASE_ARCHITECTURE.md`'s partitioning strategy should actually be triggered.
 - **Storage growth.** Tracked via Storage Monitoring above, against the file-category breakdown in `DATABASE_ARCHITECTURE.md` Section 20, to anticipate object-storage capacity and cost needs ahead of time.
 - **API scaling.** Tracked via Application/API Monitoring above, feeding into `DEPLOYMENT_ARCHITECTURE.md`'s horizontal scaling mechanics — this document's contribution is the operational trigger point (sustained approach toward a defined capacity threshold), not the scaling mechanism itself.
 - **Cost monitoring.** Infrastructure cost trends are reviewed on a defined operational cadence, correlated against the growth signals above, so a capacity or usage trend that will materially affect cost is surfaced as an operational finding rather than only appearing as a surprise on a bill.
-

Release Management

- **Versioning.** Follows the scheme defined in `DEPLOYMENT_ARCHITECTURE.md`'s Release Strategy; this document does not redefine it.
- **Release approval.** The Deployment Approval gate is owned by `DEPLOYMENT_ARCHITECTURE.md` / `TESTING_STRATEGY.md`; this document's

operational contribution is ensuring the approval is actually sought and recorded as part of the release execution checklist (Documentation section), not skipped under time pressure.

- **Rollback.** See Production Operations above — the operational decision authority for triggering one.
 - **Hotfix process.** Follows `DEPLOYMENT_ARCHITECTURE.md`'s Hotfix process (expedited but real review, no bypassed security checks); this document's operational contribution is that the on-call responder is the one who initiates a hotfix request and is responsible for confirming the expedited review actually occurred before the hotfix proceeds to Production.
 - **Change management.** For operational changes that don't fit the standard release path (e.g., an infrastructure configuration adjustment made directly against Production for an active incident), this document requires the same after-the-fact documentation and review discipline as a normal release, applied retroactively rather than skipped because the change was urgent.
-

Operational Security

Policy ownership for each item below remains with `SECURITY_ARCHITECTURE.md`; this document owns the **operational cadence and execution**:

- **Secret rotation.** `SECURITY_ARCHITECTURE.md` Section 12 owns the rotation policy and rules; this document owns confirming rotation actually happens on the defined cadence and is tracked to completion, not merely policy-permitted.
- **Access reviews.** Periodic review of who holds `admin / moderator / service` role assignments (per `MASTER_ARCHITECTURE.md` Section 5's role set), confirming each assignment still reflects Least Privilege (`SECURITY_ARCHITECTURE.md` Section 1) and revoking access no longer needed.
- **Audit reviews.** Periodic operational review of Audit Log content (not just its existence) — sampling `admin / service` actions for anything unexpected, closing the loop on `SECURITY_ARCHITECTURE.md` Section 19's requirement that audit records actually get looked at.
- **Vulnerability management.** Tracking and remediating findings from the automated security checks defined in `TESTING_STRATEGY.md`'s Security Testing and `DEPLOYMENT_ARCHITECTURE.md`'s CI/CD Security Checks, triaged by the Bug Severity scale (`TESTING_STRATEGY.md`), with Critical/High findings following the same escalation urgency as a Critical alert above.
- **Compliance monitoring.** Operational confirmation that the retention and deletion commitments in `SECURITY_ARCHITECTURE.md` Section 22 and `DATABASE_ARCHITECTURE.md` Section 15 are actually being honored in practice

(e.g., that soft-deleted records are in fact purged after their grace period), not just specified.

Documentation

- **Runbooks.** Step-by-step operational procedures for known scenarios (a specific alert firing, a specific rollback, a specific restore) — living documents distinct from this architecture specification, referenced by it but maintained as operational artifacts rather than architecture.
 - **Operational checklists.** The concrete release-execution and on-call-handoff checklists referenced throughout this document (Production Operations, Release Management) — again, operational artifacts this document requires to exist, without itself being the checklist.
 - **Incident reports.** Every Critical alert and every Security incident produces a written record — what happened, what was affected, how it was resolved — feeding both the Audit Log (for the formal record) and the Postmortem process below (for the learning).
 - **Postmortems.** Blameless, written after any Critical incident, identifying root cause and any specification gap it revealed (feeding back into `MASTER_ARCHITECTURE.md` Section 11's Change Management process, per `SECURITY_ARCHITECTURE.md` Section 23's remediation step) — a postmortem that doesn't result in either a process fix or a documentation update is considered incomplete.
-

Definition of Done

- [x] Architecture only — no implementation, no scripts, no GitHub Actions, no monitoring configuration, no infrastructure code.
 - [x] Follows `README.md` and `MASTER_ARCHITECTURE.md` exactly.
 - [x] All required sections present.
 - [x] Cross-referenced against `DATABASE_ARCHITECTURE.md`, `SECURITY_ARCHITECTURE.md`, and `DEPLOYMENT_ARCHITECTURE.md` without restating their owned content.
-

Validation Report

Internal Consistency

Every section that touches a concern owned elsewhere (RPO/RTO, secrets policy, audit log record shape, rate limits, versioning scheme) explicitly defers to its owning

document and instead defines this document's own operational-execution responsibility around it. No section restates another document's specification. No internal contradiction found across the twelve sections.

Cross-Document Consistency Check

Referenced Document	Consistency Approach
README.md	This document now fills the exact slot README.md 's index reserved for it. Confirmed consistent.
MASTER_ARCHITECTURE.md	Uses the canonical role set (Section 5) and interim approval authority (Section 5) exactly as frozen there; follows the Cross-Document Precedence model (Section 9) by deferring rather than duplicating. Confirmed consistent.
DATABASE_ARCHITECTURE.md	Log retention categories (Logging section) and capacity-planning domains (Capacity Planning section) mirror that document's Section 13 (Partitioning) and Section 15 (Retention) exactly, without redefining either. Confirmed consistent.
SECURITY_ARCHITECTURE.md	Operational Security section explicitly splits policy (owned there) from execution cadence (owned here) for secret rotation, access reviews, and audit reviews; Incident Management defers to that document's Section 23 for the security-classified decision process. Confirmed consistent.
DEPLOYMENT_ARCHITECTURE.md	Backup & Recovery, Monitoring, and Release Management sections all explicitly defer numeric targets and infrastructure mechanics to that document. Flagged: since DEPLOYMENT_ARCHITECTURE.md has not yet been through its own reconciliation pass (per this document's opening note), this cross-reference is checked against its original standalone content, which was already broadly consistent with MASTER_ARCHITECTURE.md 's later ownership assignments — no conflict found, but this should be re-confirmed once DEPLOYMENT_ARCHITECTURE.md is formally reconciled.

Risk Assessment

Area	Residual Risk	Rationale
DEPLOYMENT_ARCHITECTURE.md not yet reconciled	Medium	This document's cross-references to it are sound today but unverified against a version that has itself been checked against MASTER_ARCHITECTURE.md and DATABASE_ARCHITECTURE.md line-by-line.
RPO/RTO and log-retention numeric windows still undefined	Medium	Consistently flagged (not newly introduced here) pending MASTER_GOVERNANCE_AMENDMENT_001.md ; this document correctly avoids inventing numbers but cannot fully close operational readiness without them.
On-call staffing and escalation timing thresholds	Medium	Structure is defined (who escalates to whom, and why) but exact time windows (e.g., "unacknowledged within X minutes") are unspecified pending the same governance document.
Core monitoring/logging/alerting category model	Low	Fully specified and consistent with every governing document; no open structural gap identified.
Postmortem-to-documentation feedback loop	Low	Explicitly required as part of Definition of Done for any postmortem, closing a gap that might otherwise let incidents recur silently.

Future Operational Improvements

- Once DEPLOYMENT_ARCHITECTURE.md is reconciled, re-validate this document's cross-references to it (particularly Backup & Recovery and Release Management).
- Once MASTER_GOVERNANCE_AMENDMENT_001.md exists, assign concrete numeric values to every flagged placeholder in this document (log retention windows, DR testing cadence, escalation timing) in a single coordinated pass, consistent with the precedent already set for DATABASE_ARCHITECTURE.md and SECURITY_ARCHITECTURE.md .
- Once real production traffic exists, revisit the Business Metrics and Capacity Planning sections with actual growth data rather than the target-driven planning basis used here.

- Formally produce the runbook, checklist, and postmortem *artifacts* this document requires to exist (Documentation section) – this document specifies that they must exist and what they must contain, but does not itself create them, consistent with its architecture-only scope.

Status: Validated – Ready to Govern Operational Practice